

NAAN MUDHALVAN

PROJECT REPORT

AI FAQ ASSISTANT API

Submitted in partial fulfillment of the requirements for the Naan Mudhalvan Course

Academic Year: 2026–2027

S.No	Team	Name	Email id
1.	Team Leader	Thirisha.E	thirishaneet23@gmail.com
2.	Team Member	Ramya.M	ramyamoorthi8555@gmail.com
3.	Team Member	Sharmila.M	Sharmilamj30@gmail.com
4.	Team Member	Punitha.V	Punithavelu2007@gmail.com
5.	Team Member	Priyadharshini.S	Priyadharshini.110512@gmail.com

Table of Contents

1	Introduction
2	Problem Statement
3	Objectives
4	Technology Stack
5	System Architecture
6	Authentication
7	FAQ Management
8	AI Integration
9	Database Design
10	API Endpoints
11	Security
12	Project Development
13	Project Testing
14	Project Demonstration
15	Conclusion
16	Future Enhancements

Introduction

The AI FAQ Assistant API is a backend REST API designed to simplify the creation, management, searching, and generation of Frequently Asked Questions (FAQs).

The system combines traditional FAQ management functionality with Google Gemini AI to assist in generating FAQ content from a given topic.

The application is developed using Node.js and Express.js and uses MongoDB with Mongoose for data storage.

Problem Statement

Managing frequently asked questions manually can require considerable time and effort.

Users may also have difficulty finding relevant FAQ information quickly and creating suitable questions and answers for new topics.

The project addresses these requirements by providing a centralized API for FAQ management and AI-assisted FAQ generation.

Objectives

To develop a REST API for FAQ management.

To provide secure user authentication.

To allow users to create, read, update, and delete FAQs.

To provide FAQ searching functionality.

To organize FAQs using categories.

To integrate Google Gemini AI for FAQ generation.

To store application data using MongoDB.

To provide structured JSON API responses.

Technology Stack

Backend: Node.js, Express.js

Database: MongoDB, Mongoose

Artificial Intelligence: Google Gemini AI

Security: JSON Web Token (JWT), bcrypt

Development and Testing: Visual Studio Code, Postman / ThunderClient, GitHub

System Architecture

The application follows a layered backend architecture. An API client sends requests to the Express.js server. Authentication middleware protects private operations. Controllers process requests and communicate with MongoDB or the Google Gemini AI service depending on the requested operation.

Authentication

The application uses JWT-based authentication.

The authentication process is: (1) a user registers an account, (2) the user logs in using credentials, (3) the server verifies the credentials, (4) a JWT token is generated, and (5) the token is used to access protected API endpoints.

Passwords are protected using bcrypt.

FAQ Management

The API provides functionality for managing FAQ information.

Main operations include Create FAQ, Read FAQ, Update FAQ, Delete FAQ, Search FAQ, and Categorize FAQ.

Authenticated users can perform protected FAQ management operations.

AI Integration

Google Gemini AI is integrated into the API to provide AI-assisted FAQ generation.

The user provides a topic to the API. The request is processed by the AI service and generated FAQ content is returned through the API.

Database Design

MongoDB is used for storing application data. Mongoose is used to define schemas and communicate with MongoDB.

The project includes data structures for users and FAQs, along with the required relationships and validation.

API Endpoints

Authentication: POST /api/auth/register; POST /api/auth/login

FAQ: POST /api/faqs; GET /api/faqs; PUT /api/faqs/:id; DELETE /api/faqs/:id

FAQ Search: GET /api/faqs/search?q=configure

AI FAQ Generation: POST /api/ai/generate-faq

Protected endpoints require JWT authentication.

Security

The project uses JWT authentication, bcrypt password hashing, protected API routes, input/schema validation, and centralized error handling.

These mechanisms are used to protect private operations and maintain data integrity.

Project Development

The project was developed using a modular Node.js and Express.js backend structure.

The implementation contains components for application/server configuration, database configuration, controllers, middleware, models, routes, and AI functionality.

The source code is maintained in the GitHub repository under the Project Development phase.

Project Testing

The major API functionalities are tested using API testing tools.

Testing areas include user registration, user login, JWT authentication, FAQ creation, FAQ search, AI FAQ generation, input validation, and error handling.

Project Demonstration

The project demonstration presents the working API and demonstrates the major functionality of the system.

The demonstration should include the project name, purpose, benefits, execution/working process, and final output. The project's own demonstration video should be provided through the required Google Drive link.

Conclusion

The AI FAQ Assistant API provides a backend solution for managing Frequently Asked Questions and supporting AI-assisted FAQ generation.

The system combines REST API development, database management, authentication, FAQ operations, and Google Gemini AI integration in a single backend application.

Future Enhancements

Advanced semantic FAQ search

Improved AI-generated responses

FAQ analytics

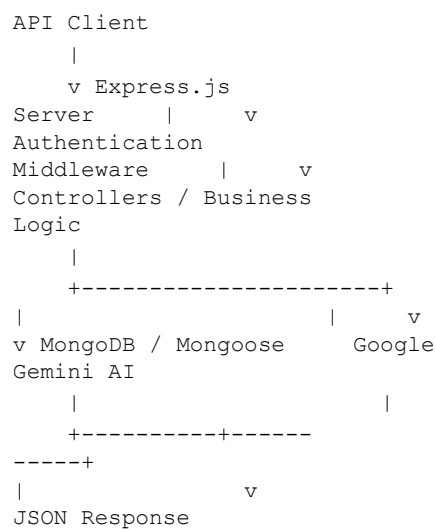
Additional user roles and permissions

API usage monitoring

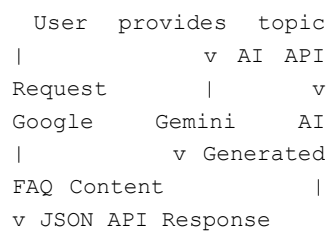
Frontend integration

Additional AI-assisted FAQ management features

System Architecture Flow



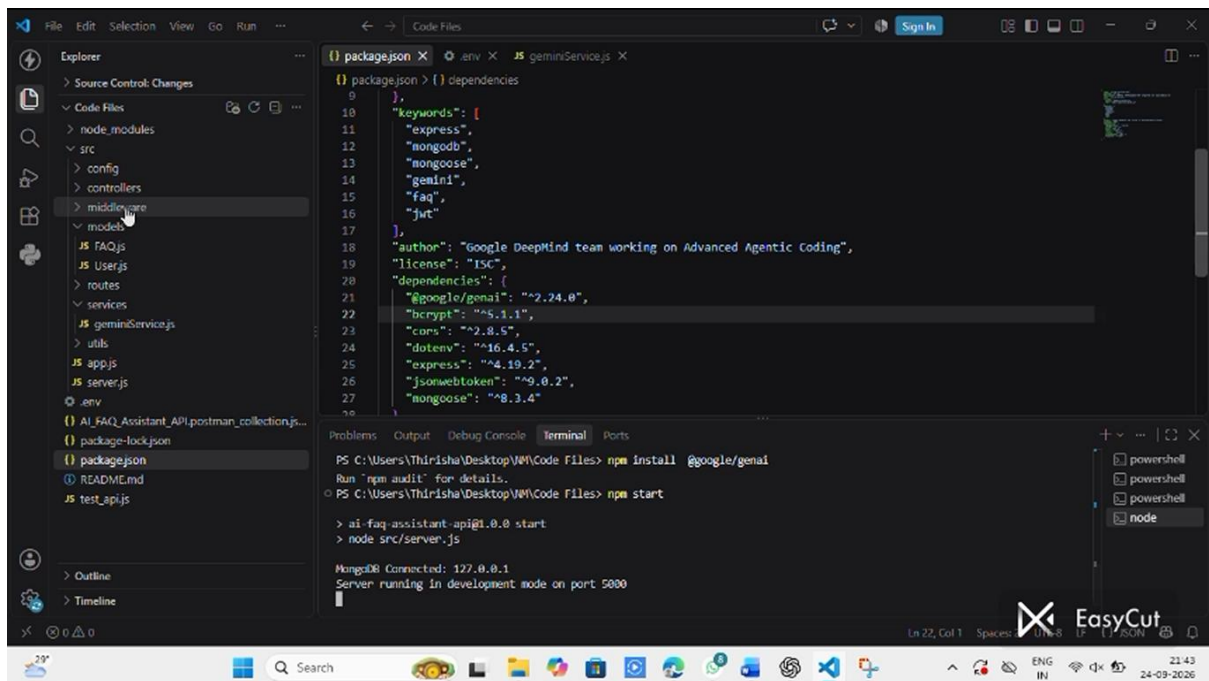
AI Workflow



Project Phase Structure

- 01_Brainstorming_and_Ideation
- 02_Requirement_Analysis
- 03_Project_Design
- 04_Project_Planning
- 05_Project_Development
- 06_Project_Testing
- 07_Project_Documentation
- 08_Project_Demonstration

OUTPUT



The screenshot displays a Visual Studio Code (VS Code) editor interface. On the left, the Explorer sidebar shows a project structure with folders like 'src', 'config', 'controllers', 'middleware', 'models', 'services', 'utils', and 'server.js'. The main editor area shows the 'package.json' file, which contains metadata and dependencies. The terminal window at the bottom shows the execution of 'npm install @google/genai' and 'npm start', followed by the output of the application running on port 5000.

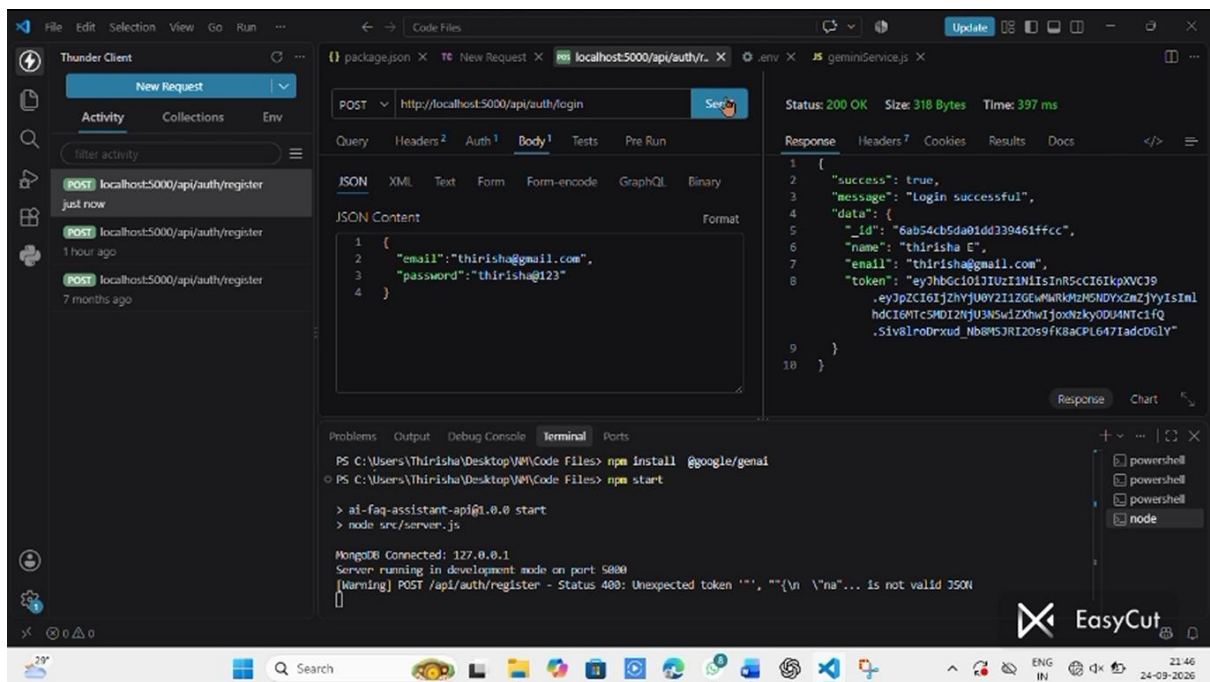
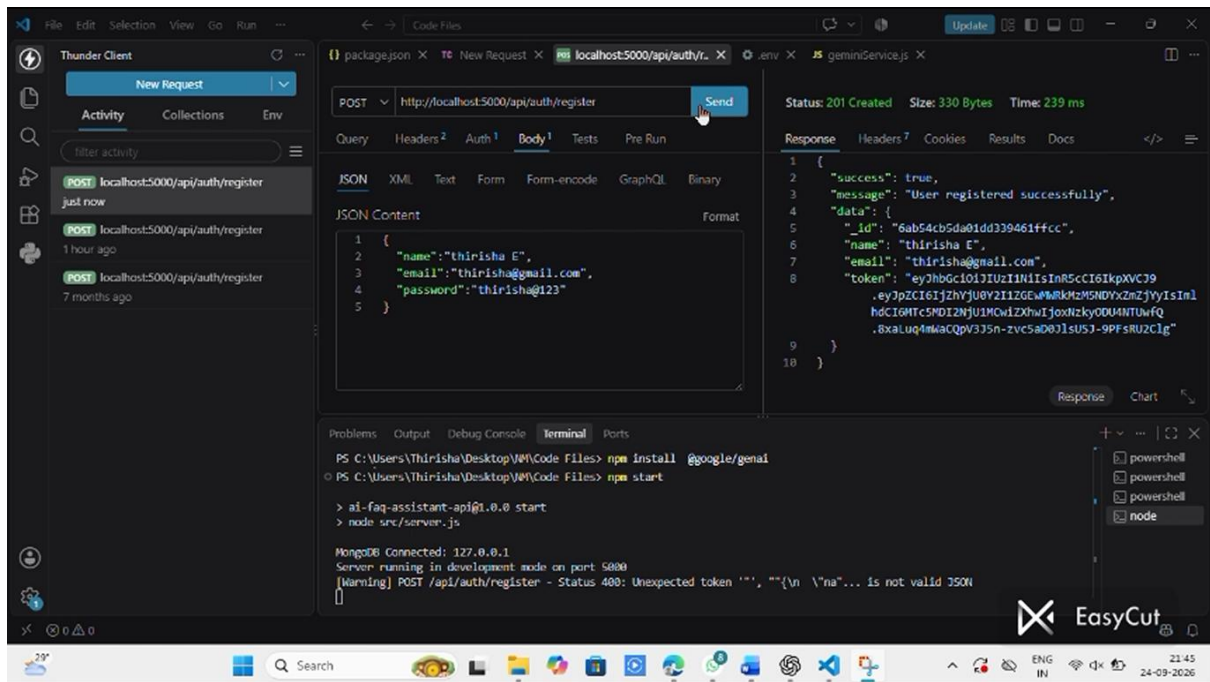
```
{} package.json > {} dependencies
{}
10  "keywords": {
11    "express",
12    "mongoose",
13    "mongoose",
14    "gemini",
15    "faq",
16    "jui"
17  },
18  "author": "Google DeepMind team working on Advanced Agentic Coding",
19  "license": "ISC",
20  "dependencies": {
21    "@google/genai": "^2.24.0",
22    "bcrypt": "^5.1.1",
23    "cors": "^2.8.5",
24    "dotenv": "^16.4.5",
25    "express": "^4.19.2",
26    "jsonwebtoken": "^9.0.2",
27    "mongoose": "^8.3.4"
28  }
29
```

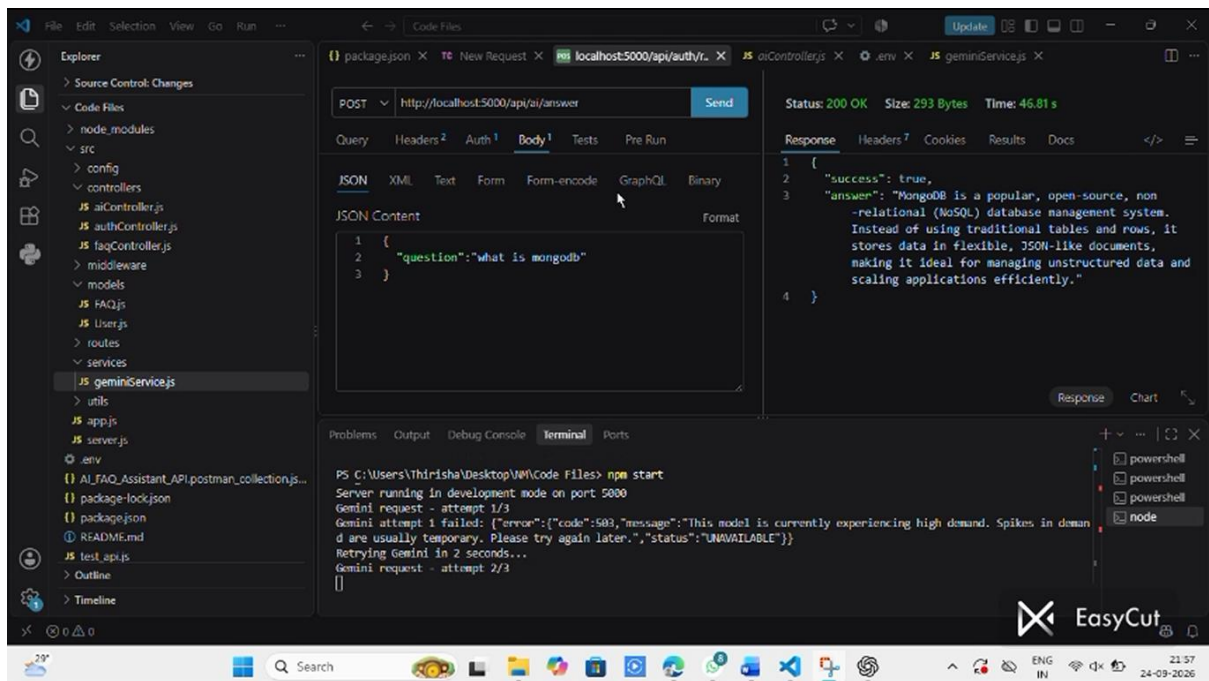
Terminal Output:

```
PS C:\Users\Thirisha\Desktop\VM\Code Files> npm install @google/genai
Run 'npm audit' for details.
PS C:\Users\Thirisha\Desktop\VM\Code Files> npm start

> ai-faq-assistant-api@1.0.0 start
> node src/server.js

MongoDB Connected: 127.0.0.1
Server running in development mode on port 5000
```





DEMO LINK

<https://drive.google.com/file/d/1A2LG2rrLHnSNZ8BNcd2ELmqhn8k6FwLZ/view?usp=sharing>